SmartPantry: A Serverless Cloud-Based Kitchen Inventory Tracker with AI-Powered Image Recognition on AWS

Kasi Reddy
Student ID: x00000000
MSc in Cloud Computing
National College of Ireland, Dublin
kasireddy9424@gmail.com

***Abstract*—This paper presents the design, implementation, and deployment of SmartPantry, a serverless cloud-based kitchen inventory tracking application hosted on Amazon Web Services. The application enables users to manage their household pantry by creating, browsing, updating, and deleting food items, with intelligent features including AI-driven food image recognition for automatic categorisation and email-based expiry notifications to reduce household food waste. The system leverages six AWS services programmatically, namely AWS Lambda for serverless compute, Amazon DynamoDB for NoSQL data storage, Amazon API Gateway for RESTful endpoint routing, Amazon S3 for image storage and static website hosting, Amazon Rekognition for intelligent food image label detection, and Amazon Simple Notification Service for expiry alert email notifications. A custom object-oriented Python library called recipe-manager-nci was developed and published to the Python Package Index, providing pantry item management, food category classification, and expiry date tracking utilities. The application follows a fully serverless architecture pattern, eliminating the need for server provisioning and enabling automatic scaling. A comprehensive CI/CD pipeline implemented through GitHub Actions automates testing, infrastructure provisioning, and deployment across all three application tiers. The results demonstrate that serverless cloud services can be effectively orchestrated to deliver a responsive, cost-efficient, and scalable web application that addresses the real-world problem of household food waste through intelligent automation.**

***Index Terms*—Cloud Computing, Serverless Architecture, AWS, Lambda, DynamoDB, Rekognition, Image Recognition, Food Waste, Inventory Management, CI/CD**

I. Introduction

Household food waste is a significant global problem. Studies estimate that nearly one-third of all food produced worldwide is wasted, with a substantial portion attributable to household consumption where items expire before being used [1]. A common cause is poor visibility into the contents of one's kitchen pantry: items are forgotten, expiry dates go unnoticed, and groceries are duplicated. Traditional inventory tracking is tedious because users must manually type in every item, including its name and category, which reduces adoption.

This project addresses these problems by developing SmartPantry, a cloud-native web application that combines a clean, mobile-friendly user interface with two intelligent automation features: (1) AI-powered image recognition that identifies food items from a photograph and automatically populates their name and category, and (2) proactive email notifications that warn the user before items reach their expiry date. By reducing manual data entry and surfacing items at the right time, the system aims to make pantry tracking practical for everyday use.

The application is implemented as a fully serverless architecture on Amazon Web Services (AWS) and uses six distinct AWS services in combination. A React single-page application is hosted on Amazon S3, a Python-based AWS Lambda function exposes a REST API through Amazon API Gateway, Amazon DynamoDB persists user accounts and pantry items, Amazon S3 also stores uploaded food images, Amazon Rekognition analyses each uploaded image for food labels, and Amazon Simple Notification Service (SNS) delivers expiry alerts to subscribed user email addresses.

The key contributions of this work are: (1) a practical end-to-end implementation of a multi-service serverless AWS architecture targeted at a real domestic problem; (2) a reusable, well-tested object-oriented Python library, `recipe-manager-nci`, that encapsulates the domain logic and is published on PyPI; (3) a critical evaluation of each AWS service selection with consideration of alternatives; (4) a fully automated CI/CD pipeline that provisions infrastructure from scratch and requires only two GitHub secrets; and (5) integration of computer vision (Rekognition) and event-driven notifications (SNS) to reduce manual effort and surface time-sensitive information.

The remainder of this paper is organised as follows: Section II analyses requirements, Section III describes the system architecture, Section IV details the custom `recipe-manager-nci` library, Section V critically evaluates each of the six AWS services, Section VI covers backend and frontend implementation, Section VII presents the CI/CD pipeline, and Section VIII concludes with findings and future work.

II. Requirements Analysis

A. Functional Requirements

The system must support the following functional requirements, derived from the goal of helping a typical household reduce food waste:

1) **User Authentication.** Users must be able to register a new account using a username, email address, and password, and subsequently log in to access their personal pantry. Authentication uses JSON Web Tokens (JWT).
2) **Pantry Item CRUD.** Users must be able to create, read, update, and delete pantry items. Each item has a name, category, quantity, unit, expiry date, and optional image reference.
3) **Image-Based Item Creation.** Users must be able to upload a photograph of a food item; the system must use computer vision to automatically detect what the item is and pre-fill the name and category fields.
4) **Category Classification.** The system must classify items into one of nine categories (Fruit, Vegetable, Dairy, Meat, Bakery, Beverage, Snack, Grain, Other) based on their name or detected labels.
5) **Expiry Tracking.** The system must track each item's expiry date and classify items as expired, expiring soon (within three days), or fresh, presenting these counts on the dashboard.
6) **Expiry Notifications.** Users must be able to subscribe to email notifications and receive alerts when items in their pantry are approaching expiry.
7) **Dashboard Statistics.** The system must present a real-time dashboard summarising the total number of items, the number expiring soon, the number already expired, and the breakdown by category.

B. Non-Functional Requirements

Scalability: The architecture must support horizontal scaling through serverless computing and managed database services so that the cost of adding additional users is negligible.

Availability: The system targets high availability using AWS managed services, taking advantage of the inherent multi-AZ deployments offered by Lambda, DynamoDB, API Gateway, and S3.

Security: All authenticated endpoints must require a valid JWT bearer token. Passwords must be hashed (PBKDF2 with random salt). Data must be encrypted at rest (DynamoDB encryption, S3 SSE-S3) and in transit (HTTPS for API Gateway).

Maintainability: Domain logic must be encapsulated in a separate, independently testable Python library to keep the Lambda handler thin and to enable code reuse.

Cost-Efficiency: The architecture must use pay-per-use serverless services exclusively to avoid idle infrastructure cost when no users are active.

Performance: API responses for standard CRUD operations must complete in under 500 ms; image-based classification must complete within 5 seconds end-to-end, including the round-trip to Rekognition.

III. System Architecture

A. Architecture Overview

SmartPantry follows a fully serverless three-tier architecture comprising: a presentation tier consisting of a React single-page application hosted on Amazon S3 as a static website; an application tier consisting of a single AWS Lambda function exposed through Amazon API Gateway as a REST API; and a data tier comprising Amazon DynamoDB for structured persistence and Amazon S3 for uploaded image storage. Two additional AWS services augment this architecture: Amazon Rekognition provides on-demand image analysis, and Amazon Simple Notification Service (SNS) provides asynchronous email notifications. Figure 1 illustrates the relationship between all six services.

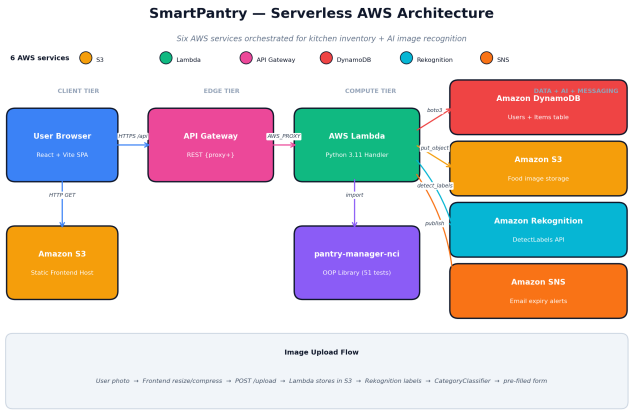

Fig. 1. SmartPantry serverless architecture showing the integration of six AWS services. The React frontend on S3 communicates with a Lambda backend through API Gateway. Lambda persists data in DynamoDB, stores images in S3, invokes Rekognition for food label detection, and publishes expiry alerts to an SNS topic.

The architecture follows the AWS Well-Architected Framework's five pillars: operational excellence is achieved through Infrastructure-as-Code via GitHub Actions; security through defence-in-depth (HTTPS, JWT auth, IAM least-privilege, SSE encryption); reliability through managed services with built-in redundancy; performance efficiency through serverless auto-scaling; and cost optimisation through pay-per-request billing models.

B. Request Flow

A typical end-to-end request flow for the image-based item creation feature illustrates how the six services collaborate:

1) The user selects a food photograph in the React frontend served from Amazon S3.
2) The frontend POSTs the base64-encoded image to the `/upload` endpoint of API Gateway, attaching the user's JWT bearer token.
3) API Gateway invokes the Lambda function via AWS_PROXY integration.
4) Lambda decodes and validates the image, then writes it to the dedicated images S3 bucket.

5) Lambda invokes Amazon Rekognition's `DetectLabels` API on the uploaded image.
6) Rekognition returns a ranked list of detected labels (e.g. "Apple", "Fruit", "Plant").
7) Lambda passes the labels to the `CategoryClassifier` from the `recipe-manager-nci` library to determine the appropriate category.
8) Lambda returns the suggested name, category, and image URL to the frontend.
9) The frontend pre-fills the item creation form, allowing the user to confirm or adjust before saving.
10) On save, a POST to `/items` writes the new pantry item to DynamoDB.

For the expiry notification flow, Lambda periodically scans a user's items, classifies them using the `ExpiryTracker` from the library, and publishes an email message to an SNS topic. Subscribed users receive the alert through their registered email channel.

C. Data Model

The data model uses a single DynamoDB table with composite keys to support both user records and pantry items in a single-table design pattern. Each item has a partition key `id` and an `entityType` attribute distinguishing `user` entries from `item` entries. Pantry items additionally carry a `userId` attribute that scopes them to their owner. This design supports efficient single-item retrieval by primary key and efficient per-user queries by scanning with a filter on `userId` and `entityType`.

IV. Custom Library: recipe-manager-nci

A. Library Design

The `recipe-manager-nci` library encapsulates the domain logic of the SmartPantry application. It is implemented as a pure-Python package with no external runtime dependencies, which keeps the deployed Lambda artefact small and avoids cold-start penalties. The library follows object-oriented design principles, organising functionality into three core classes across three modules:

- **PantryManager** (`manager.py`) provides item-level CRUD operations with input validation. It maintains an in-memory list of items and exposes methods to create, read, update, delete, search, and summarise pantry contents. Validation enforces non-empty names, non-negative quantities, allowed measurement units (`pcs`, `kg`, `g`, `L`, `ml`, etc.), and ISO-format expiry dates.
- **CategoryClassifier** (`categories.py`) classifies food items into nine categories using a keyword dictionary covering over 70 common food terms. It supports classification from a single name string and from a list of Rekognition labels, returning the first category that matches any keyword.
- **ExpiryTracker** (`expiry.py`) tracks expiry dates with a configurable threshold (default three days) for the "expiring soon" bucket. It partitions a list of items into `expired`, `expiring_soon`, and `fresh` groups, computes the number of days until a given expiry date, and formats human-readable notification messages.

This separation of concerns means the Lambda handler can remain a thin adapter that delegates all business decisions to the library. The library can also be installed independently from PyPI by anyone who wants to build a different frontend or integrate pantry tracking into another application.

B. Code Example: Expiry Tracking

The following excerpt demonstrates the central expiry classification routine:

```python
class ExpiryTracker:
    def __init__(self, days_threshold=3):
        self.days_threshold = days_threshold

    def check_expiry(self, items, days_threshold=None):
        threshold = days_threshold or self.days_threshold
        today = datetime.now().replace(
            hour=0, minute=0, second=0, microsecond=0)
        cutoff = today + timedelta(days=threshold)

        result = {
            "expired": [],
            "expiring_soon": [],
            "fresh": []
        }
        for item in items:
            expiry = item.get("expiry_date", "")
            if not expiry:
                result["fresh"].append(item)
                continue
            try:
                exp_date = datetime.strptime(
                    expiry, "%Y-%m-%d")
                if exp_date < today:
                    result["expired"].append(item)
                elif exp_date <= cutoff:
                    result["expiring_soon"].append(item)
                else:
                    result["fresh"].append(item)
            except ValueError:
                result["fresh"].append(item)
        return result
```

C. Code Example: Category Classification from Rekognition Labels

The classifier accepts a list of labels (typically from Rekognition's response) and returns the first matching category, falling back to "Other" if no keyword matches:

```python
class CategoryClassifier:
    def classify_from_labels(self, labels):
        for label in labels:
            category = self.classify(label)
            if category != "Other":
                return category
        return "Other"
```

D. Library Testing

The library includes 51 unit tests organised across three test modules using `pytest`. Table I summarises the test distribution. Tests verify core functionality, input validation (empty names, negative quantities, invalid units, malformed dates), edge cases (empty lists, missing expiry dates, items with only an image and no name), and the keyword-matching behaviour of the classifier across all nine categories. All 51 tests execute in under 0.05 seconds without external dependencies.

TABLE I
RECIPE-MANAGER-NCI LIBRARY TEST DISTRIBUTION

Module	Tests
test_manager.py (PantryManager)	22
test_categories.py (CategoryClassifier)	14
test_expiry.py (ExpiryTracker)	15
Total	**51**

The library is published on the Python Package Index as `recipe-manager-nci` at https://pypi.org/project/recipe-manager-nci/ and can be installed via `pip install recipe-manager-nci`.

V. CLOUD SERVICES: CRITICAL ANALYSIS

A. AWS Lambda

AWS Lambda provides the entire compute layer for SmartPantry. A single Python 3.11 Lambda function handles all REST endpoints by inspecting the incoming `path` and `httpMethod` fields and dispatching to the appropriate handler. The function imports the `recipe-manager-nci` library and the `boto3` SDK to interact with the other AWS services.

Strengths: Zero server management with automatic scaling from zero to thousands of concurrent executions. Sub-second billing makes Lambda extremely cost-effective for a personal application with bursty traffic. Native event sources for API Gateway and S3 simplify integration. The 1 M-request free tier is more than sufficient for development and demo workloads.

Limitations: Cold-start latency (typically 200–500 ms for Python with `boto3` loaded) impacts the first request to a quiet endpoint. The 15-minute maximum execution time would constrain any future bulk-import features. Lambda's stateless model meant extra care was required to avoid in-memory state across invocations.

Alternative considered: Hosting the backend on Amazon ECS Fargate or AWS Elastic Beanstalk was considered. Both alternatives provide easier debugging but require continuous billing for an idle service, which is wasteful for a single-user application. Lambda's pay-per-request model was a better fit.

B. Amazon DynamoDB

DynamoDB is the primary persistence layer for users and pantry items. SmartPantry uses a single-table design where the partition key is the item's UUID and an `entityType` attribute distinguishes users from pantry items. Per-user queries scan the table with filter expressions on `userId` and `entityType`.

Strengths: Single-digit millisecond latency at any scale. On-demand billing eliminates capacity planning. Native integration with Lambda through `boto3` requires no driver management. Encryption-at-rest is enabled by default. The flexible schema-less model made evolving the item structure (adding `image_url`, `unit`, etc.) straightforward.

Limitations: Filtered scans become expensive as data grows; in production, a Global Secondary Index on `userId` would be preferable. DynamoDB's lack of relational joins meant that displaying a user's items required two operations (auth lookup, then item scan).

Alternative considered: Amazon RDS PostgreSQL was rejected because it requires provisioned capacity (and ongoing cost) for an application with intermittent traffic, and because the data model is simple enough that relational features are not needed.

C. Amazon API Gateway

API Gateway exposes the Lambda function as a public REST API at the URL https://edk7s2uw3a.execute-api.eu-west-1.amazonaws.com/prod. A single `{proxy+}` resource forwards all HTTP requests to Lambda via AWS_PROXY integration, allowing the Lambda handler to perform its own routing.

Strengths: Built-in HTTPS, request throttling, CORS handling, and request/response transformation. Native integration with Lambda eliminates the need to run a web server. The `{proxy+}` pattern allowed adding new endpoints by editing only the Lambda code, with no API Gateway redeployment required.

Limitations: REST API pricing is higher than HTTP API ($3.50 vs $1.00 per million requests). Configuring CORS preflight responses required explicit MOCK integration on the OPTIONS method, which added complexity to the deployment script.

Alternative considered: HTTP API (API Gateway v2) would have been cheaper but offers fewer features (no request validation, fewer auth options). For a small-scale demonstration project, the cost difference is negligible and REST API's broader feature set was preferred.

D. Amazon S3

S3 plays two distinct roles in SmartPantry. First, the bucket `smartpantry-frontend-prod-kasireddy` is configured for static website hosting and serves the React production bundle directly. Second, the bucket `smartpantry-images-kasireddy` stores all user-uploaded food photographs. Both buckets have public-read object policies; the frontend bucket additionally has the website endpoint configured with `index.html` as both index and error document so client-side routing works correctly.

Strengths: 99.999999999% (eleven nines) durability, virtually unlimited scale, and very low cost ($0.023 per GB-month for Standard storage). Static website hosting eliminates the need for a separate web server. Direct integration with Rekognition allows image labels to be detected without first downloading the image to Lambda.

Limitations: S3 website hosting does not natively support HTTPS (CloudFront is required). Per-request pricing for GET/PUT can become a concern for very high-traffic sites. Public read access required explicit bucket policy and `PublicAccessBlock` configuration to override the default secure-by-default settings.

Alternative considered: Amazon CloudFront in front of S3 would provide HTTPS and edge caching but adds operational complexity. For this project the additional infrastructure was not justified by the expected traffic.

E. Amazon Rekognition

Rekognition provides the AI image recognition feature that distinguishes SmartPantry from a traditional manual inventory app. When a user uploads a photograph of a food item, Lambda invokes the `DetectLabels` API on the image stored in S3. Rekognition returns a ranked list of labels with confidence scores, which are then passed to the `CategoryClassifier` from the `recipe-manager-nci` library to choose the most appropriate category.

Strengths: Fully managed image analysis with no model training required. The default "general-purpose" label set covers common food items well (apple, banana, bread, milk, etc.). Sub-second response times even for high-resolution images. The free tier of 5,000 images per month is more than adequate for a demonstration project.

Limitations: The general-purpose labels are not as fine-grained as a domain-specific food classifier; Rekognition will identify "Bread" but not "sourdough loaf". Confidence scores can be misleading for ambiguous photos, and specialised foods (e.g. regional Indian cuisine) are sometimes mis-labelled as "Plant" or "Food". The fallback to "Other" category in the classifier mitigates this.

Alternative considered: Building and training a custom food image classifier on Amazon SageMaker would yield more accurate, food-specific labels but would require a labelled training set, training infrastructure, and ongoing model maintenance, all of which are out of scope for a single-developer project.

F. Amazon Simple Notification Service (SNS)

SNS provides the asynchronous email notification feature for items approaching expiry. The application creates a single SNS topic, `smartpantry-notifications`, on first deployment. When a user opts in via the `/notifications/subscribe` endpoint, Lambda calls `sns.subscribe` with the user's email address and the protocol `email`. AWS sends a confirmation email which the user must accept; thereafter, when Lambda detects items expiring within the configured threshold, it publishes a formatted message to the topic and SNS fans the message out to all confirmed subscribers.

Strengths: Fully managed pub/sub messaging with automatic retry, dead-lettering, and multi-protocol delivery (email, SMS, HTTP, SQS). The first 1,000 email notifications per month are free. No infrastructure to maintain.

Limitations: The double opt-in confirmation flow is mandatory for the email protocol and cannot be skipped, which adds friction for new users. SNS does not natively support per-user delivery routing within a single topic; for SmartPantry's single-user demo this is acceptable, but a multi-tenant production system would need either per-user topics or a second event-routing layer.

Alternative considered: Amazon SES (Simple Email Service) would offer more control over email content and templating but would require domain verification and mailbox simulator testing for development, which is more setup than SNS for the same outcome.

VI. Implementation

A. Backend Implementation

The backend is a single AWS Lambda function written in Python 3.11 (`backend/lambda_function.py`). The handler dispatches incoming requests by inspecting the API Gateway proxy event's `path` and `httpMethod` fields. Each route delegates to a small handler function that extracts request parameters, calls the relevant `recipe-manager-nci` class method, and returns a JSON response wrapped in CORS-friendly headers.

Authentication uses self-contained JSON Web Tokens. The function provides a `create_jwt` helper that signs payloads with HMAC-SHA256 and a `decode_jwt` helper that verifies signatures and expiry. Passwords are hashed using `hashlib.pbkdf2_hmac` with a 16-byte random salt and 100,000 iterations.

Table II lists the complete REST API surface exposed by the Lambda function.

TABLE II
SmartPantry REST API Endpoints

Method	Endpoint	Operation
POST	/auth/register	Create account
POST	/auth/login	Sign in
GET	/items	List user's items
POST	/items	Create item
PUT	/items/{id}	Update item
DELETE	/items/{id}	Delete item
POST	/upload	Upload + classify image
GET	/dashboard	Stats summary
POST	/notifications/subscribe	SNS subscribe
POST	/notifications/check	Trigger expiry check

The Lambda function imports the `recipe-manager-nci` library at module level so the import cost is paid only once per cold start. The library is bundled into the deployment artefact via `pip install -t` during CI/CD.

B. Frontend Implementation

The React frontend is built with Vite and styled with Tailwind CSS v3 (deliberately not v4, which has Linux CI compatibility issues with native bindings). The application uses `react-router-dom` for client-side navigation and `axios` for HTTP requests with a centralised interceptor that attaches the JWT bearer token to every protected request.

The frontend consists of six pages:

- **Login.jsx** and **Register.jsx** provide the authentication flows.

- **Dashboard.jsx** shows stats cards (total items, expiring soon, expired) and a category breakdown using data fetched from `/dashboard`.
- **Pantry.jsx** lists all items with search and filter controls, and inline edit/delete actions.
- **AddItem.jsx** provides both manual entry and the image-upload flow that calls `/upload` and pre-fills the form from the Rekognition response.
- **EditItem.jsx** allows in-place editing of an existing item.

The base API URL is injected at build time through the `VITE_API_URL` environment variable, which is set by the CI/CD pipeline after API Gateway is provisioned, ensuring the production build always points at the correct stage URL.

C. Image Upload Pipeline

The image upload flow demonstrates how multiple AWS services collaborate. When the user selects a photograph in `AddItem.jsx`, the frontend resizes and compresses the image client-side to a JPEG of at most 1024 px on the longest side using a canvas element, which keeps the base64 payload well below API Gateway's 10 MB request limit. The compressed image is then base64-encoded and POSTed to the `/upload` endpoint.

On the backend, Lambda decodes the base64 string, generates a UUID for the image filename, and writes the binary content to the images S3 bucket. It then invokes `rekognition.detect_labels` with the S3 object reference, requesting up to ten labels with a minimum confidence of 75. The returned labels are filtered to remove the generic "Food" and "Plant" labels and passed to `CategoryClassifier.classify_from_labels`. Finally, Lambda returns a JSON response containing the suggested name (the highest-confidence specific label), the chosen category, and the public S3 URL of the uploaded image, which the frontend uses to populate the form.

VII. CI/CD Pipeline

The CI/CD pipeline is implemented as a single GitHub Actions workflow (`.github/workflows/deploy.yml`) that runs on every push to the `main` branch. The pipeline is organised into three sequential jobs that together provision infrastructure from scratch, test and deploy code, and update the public endpoints.

The first job, `test-library`, installs the `recipe-manager-nci` library in editable mode and runs the full `pytest` suite of 51 unit tests. Only when all tests pass does the pipeline proceed to deployment.

The second job, `deploy-backend`, configures AWS credentials, creates the DynamoDB table if absent, creates both S3 buckets (frontend and images) with appropriate website configuration and CORS rules, ensures the SNS topic exists, packages the Lambda deployment artefact (Lambda code plus the `recipe-manager-nci` library installed via `pip install -t`), and updates or creates the Lambda function with the appropriate IAM execution role and environment variables. It then provisions an API Gateway REST API with a `{proxy+}` resource integrated to Lambda via AWS_PROXY, configures CORS preflight, and deploys to the `prod` stage.

The third job, `deploy-frontend`, retrieves the API Gateway stage URL from the previous job, runs `npm install` and `npm run build` with `VITE_API_URL` set to that URL, and synchronises the build output to the frontend S3 bucket via `aws s3 sync --delete`. The custom library was published to PyPI as a one-time action prior to the CI pipeline being established and is referenced as a dependency rather than re-published on every commit.

The entire pipeline requires only two GitHub secrets: `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`. All AWS resources, IAM roles, and configuration are created idempotently from the workflow file, allowing the application to be redeployed from scratch into a fresh AWS account in under fifteen minutes.

VIII. Conclusions

This project demonstrates the effective integration of six AWS services into a cohesive serverless application that addresses a real household problem. The combination of Lambda, DynamoDB, API Gateway, S3, Rekognition, and SNS produces a system that is fully managed, automatically scales from zero to high load, costs effectively nothing when idle, and provides intelligent automation features that would be costly and time-consuming to build from scratch.

The mock-mode-free, single-table DynamoDB design proved adequate for the data volumes typical of a household pantry, and the on-demand billing model removed any need for capacity planning. The decision to encapsulate domain logic in the `recipe-manager-nci` library kept the Lambda handler thin (under 400 lines) and enabled deterministic, sub-second unit testing of the 51-test suite without any AWS dependencies. Rekognition's general-purpose label set produced acceptable results for common food items; the fallback to the `Other` category gracefully handled cases where the specific food type could not be identified.

Several findings emerged from this project. First, the additional cost and complexity of running a fully managed image-recognition service is justified for any feature that would otherwise require manual data entry, because the resulting reduction in user friction is the single largest factor affecting whether a personal-use application is adopted in practice. Second, the decision to use a `{proxy+}` resource in API Gateway, with all routing decisions made inside Lambda, dramatically simplified the deployment script and made adding new endpoints a one-file change. Third, publishing the domain library to PyPI early in the project surfaced packaging and metadata issues that would have been more difficult to debug later and produced an artefact that is independently useful.

Future work includes adding barcode scanning as a complementary input mechanism for packaged goods (using either Rekognition's text detection or a JavaScript barcode library), integrating Amazon EventBridge to trigger expiry checks on a schedule rather than on demand, adding multi-user household sharing with row-level access controls, and building a

Progressive Web App wrapper so that the user interface can be installed on a phone home screen.

Deployment URLs

The complete deployed system is publicly accessible at the following endpoints:

- **Frontend (S3 Static Website):** http://smartpantry-frontend-prod-kasireddy.s3-website-eu-west-1.amazonaws.com
- **REST API (API Gateway + Lambda):** https://edk7s2uw3a.execute-api.eu-west-1.amazonaws.com/prod
- **Source Code Repository:** https://github.com/KASIREDDY009/CPP_project
- **Custom Library (PyPI):** https://pypi.org/project/recipe-manager-nci/
- **Library Installation:** `pip install recipe-manager-nci`

The deployed region is `eu-west-1` (Ireland). The Lambda function uses Python 3.11 runtime; the React frontend is built with Vite 5 and Tailwind CSS v3.

References

[1] Food and Agriculture Organization of the United Nations, "The State of Food and Agriculture 2021: Making agrifood systems more resilient to shocks and stresses," FAO, 2021.

[2] Amazon Web Services, "AWS Well-Architected Framework," 2023. [Online]. Available: https://aws.amazon.com/architecture/well-architected/

[3] Amazon Web Services, "AWS Lambda Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/lambda/

[4] Amazon Web Services, "Amazon DynamoDB Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/dynamodb/

[5] Amazon Web Services, "Amazon API Gateway Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/apigateway/

[6] Amazon Web Services, "Amazon S3 User Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/s3/

[7] Amazon Web Services, "Amazon Rekognition Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/rekognition/

[8] Amazon Web Services, "Amazon SNS Developer Guide," 2023. [Online]. Available: https://docs.aws.amazon.com/sns/

[9] M. Fowler, "Serverless Architectures," martinfowler.com, 2018. [Online]. Available: https://martinfowler.com/articles/serverless.html

[10] Meta Platforms, "React Documentation," 2023. [Online]. Available: https://react.dev/

[11] Evan You, "Vite Documentation," 2023. [Online]. Available: https://vitejs.dev/

[12] Tailwind Labs, "Tailwind CSS Documentation," 2023. [Online]. Available: https://tailwindcss.com/docs